BUILDING EVOLUTIONARY ARCHITECTURES

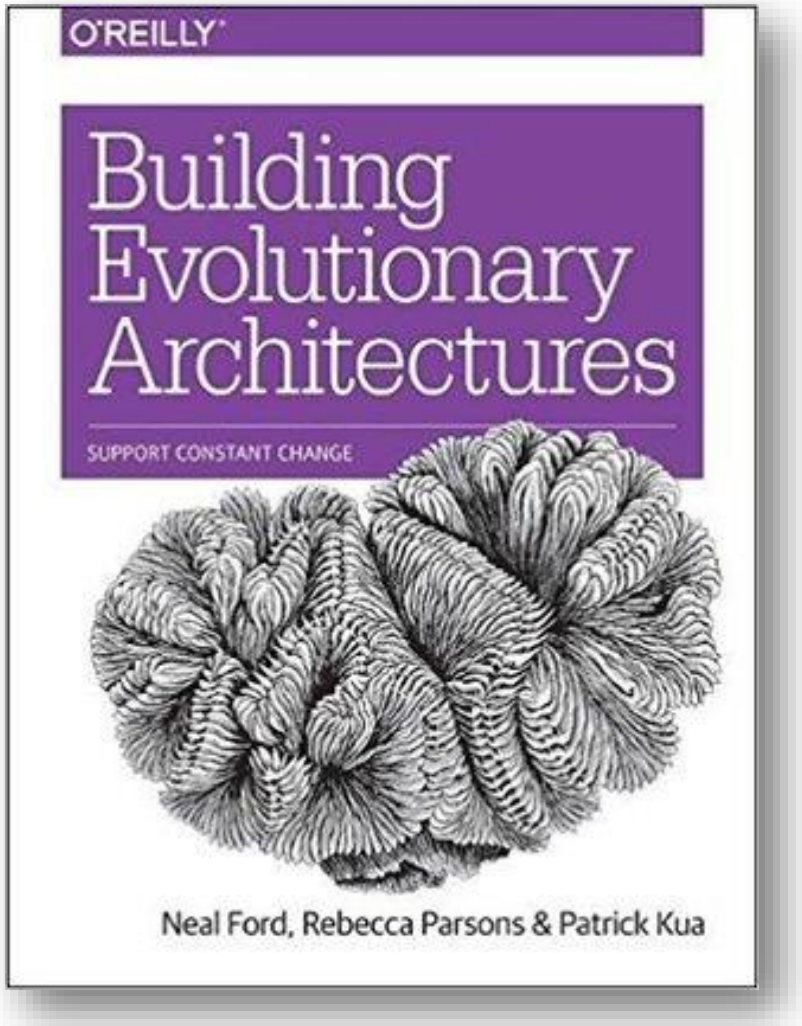
Krakow, 9-11 May 2018

THE FIGHTERS

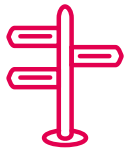
Xavier RENÉ-CORAIL
@xcorail

Ionuţ BALOŞIN
@ionutbalosin

INSPIRATIONS

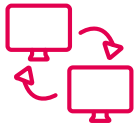


AGILE & ARCHITECTURE



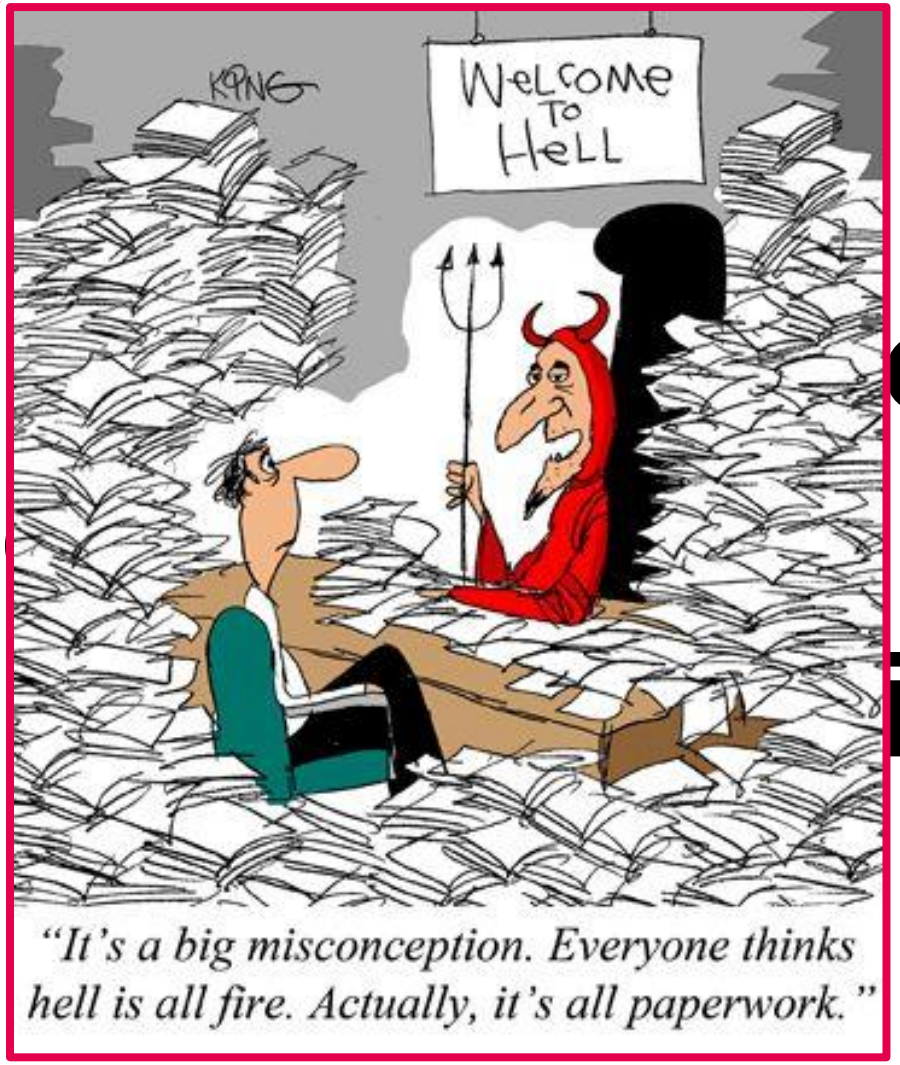
ENEMIES ? …

… OR FRIENDS ?



WORKING TOGETHER

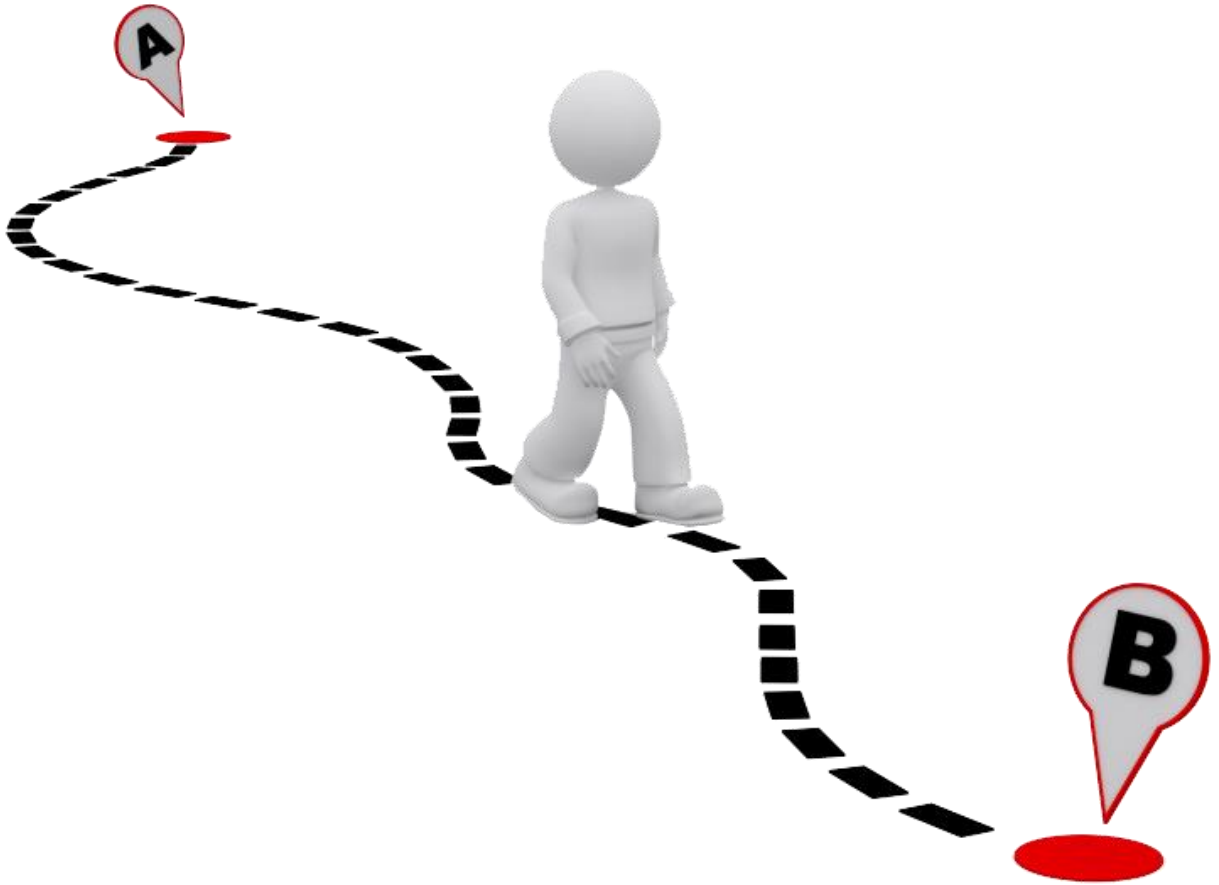
MISCONCEPTIONS



ome

ions

RESPONDING TO CHANGE

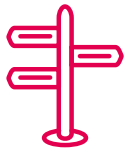


BE RIGHT THE 1ST TIME

Is there one thing

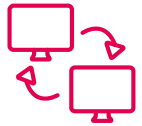
that will never change?

AGILE & ARCHITECTURE



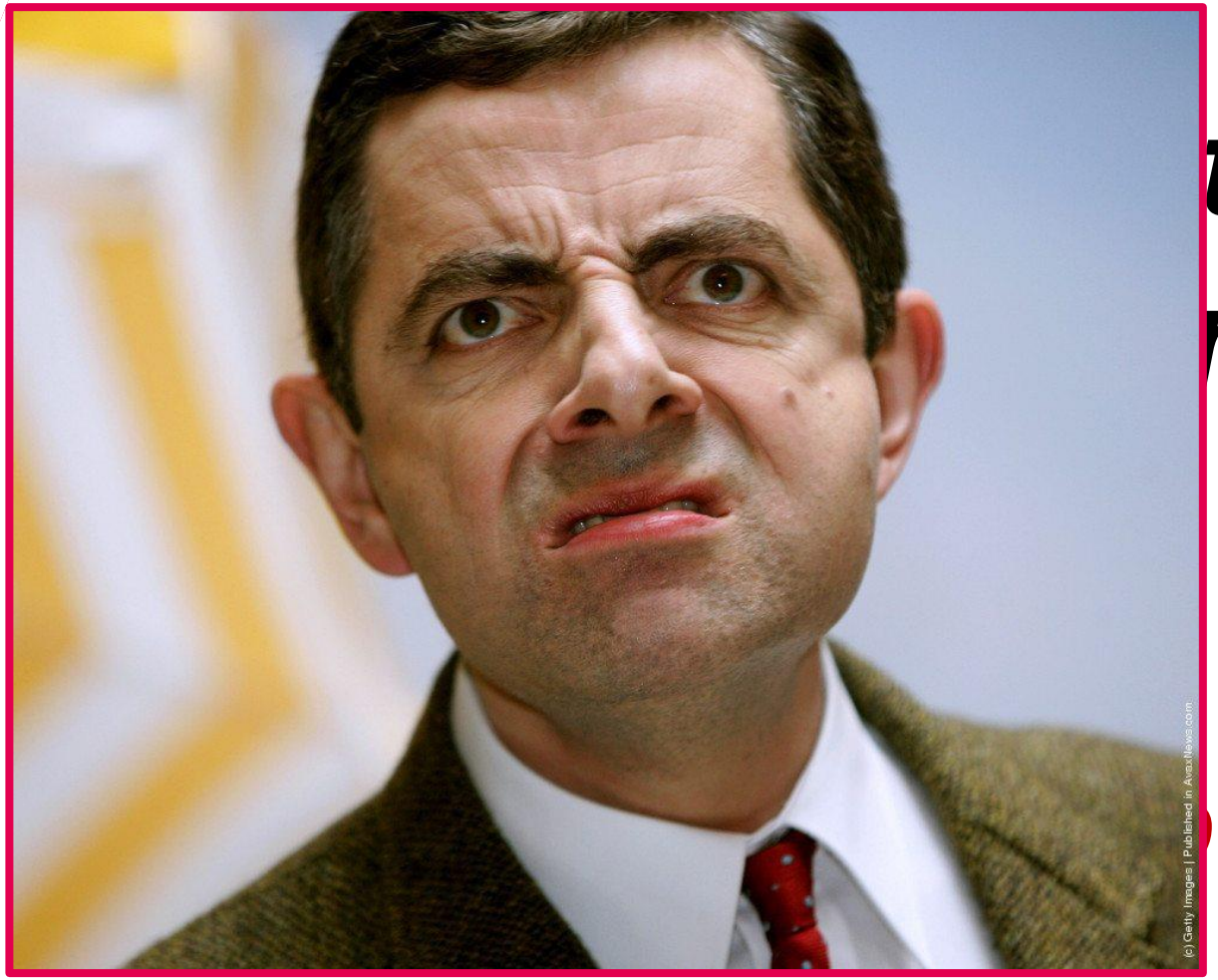
ENEMIES ? …

… OR FRIENDS ?



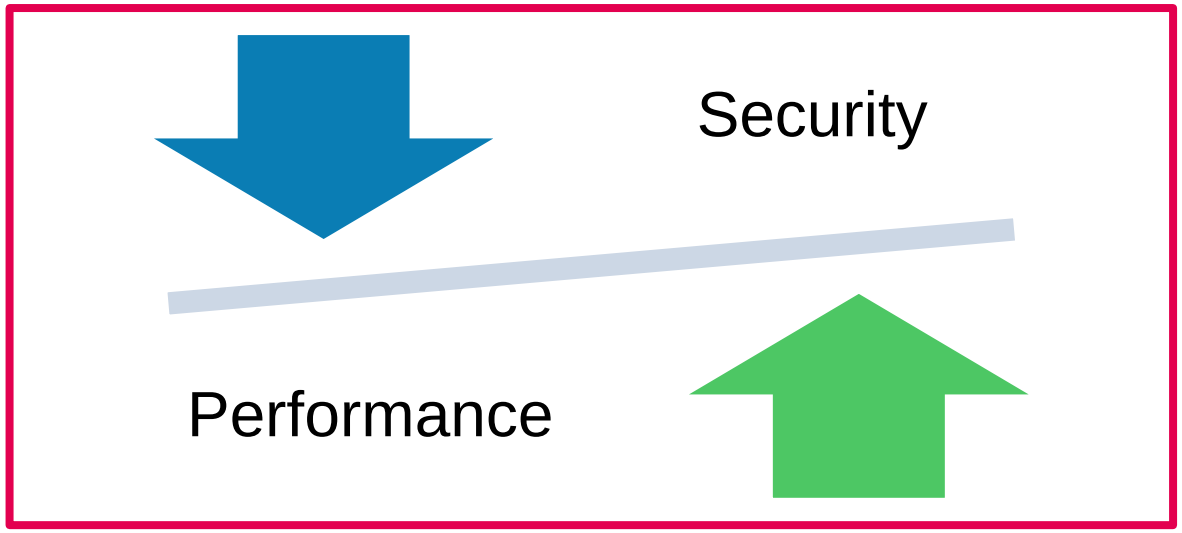
WORKING TOGETHER

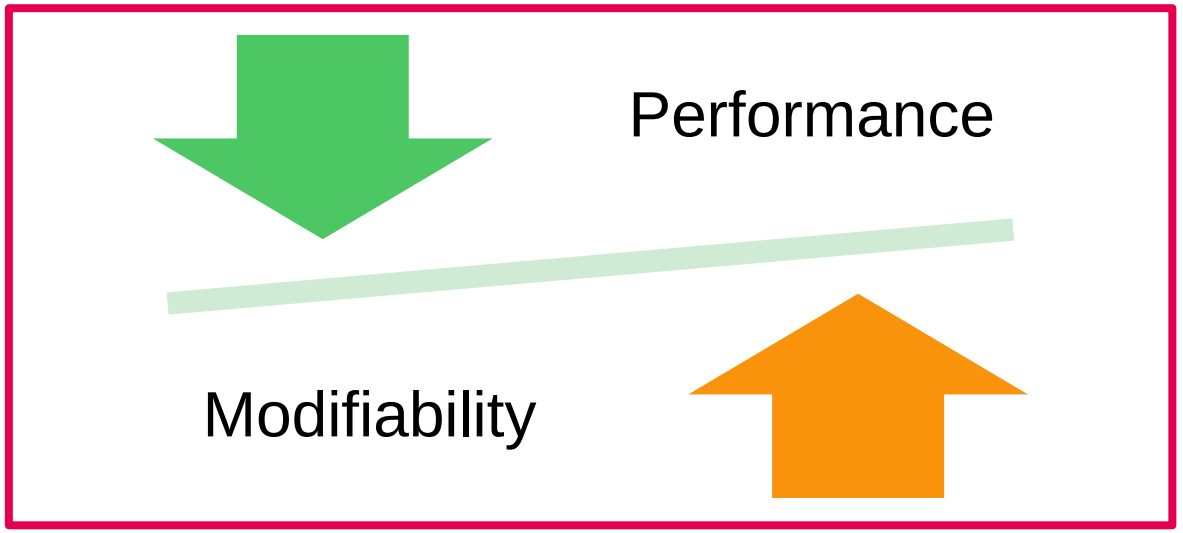
SOFTWARE ARCHITECTURE IS ...



WORLD OF '-ILITIES

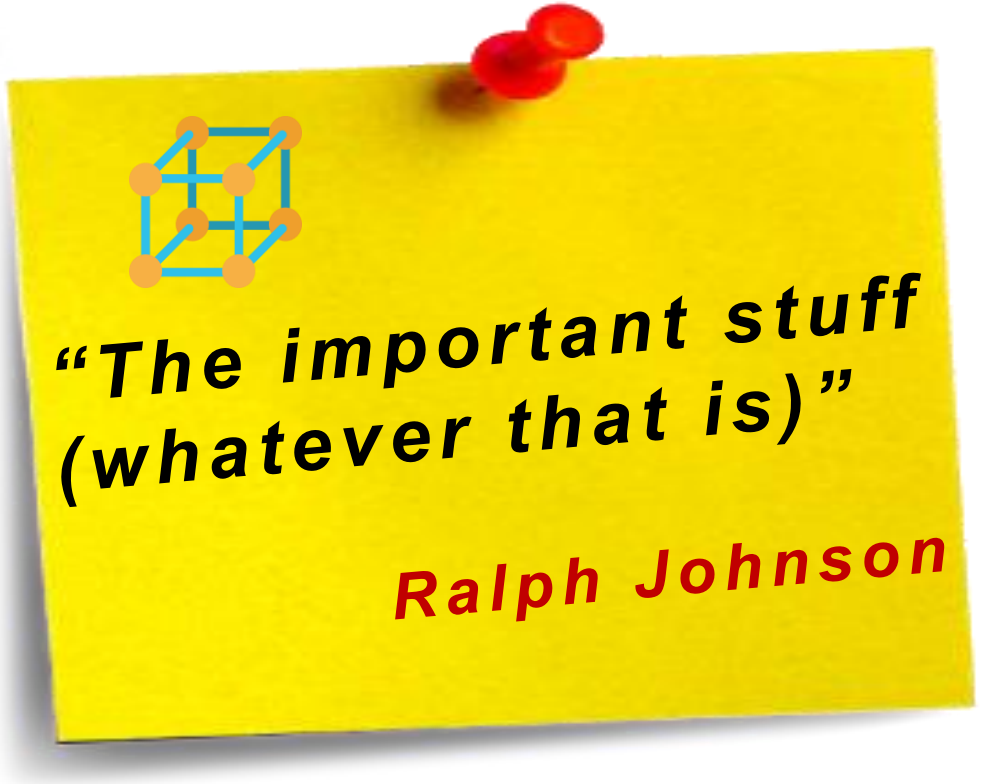
DIVERGENT '-ILITIES





WE NEED TO ELICIT

SOFTWARE ARCHITECTURE IS ...



STAKEHOLDER PRIORITISATION

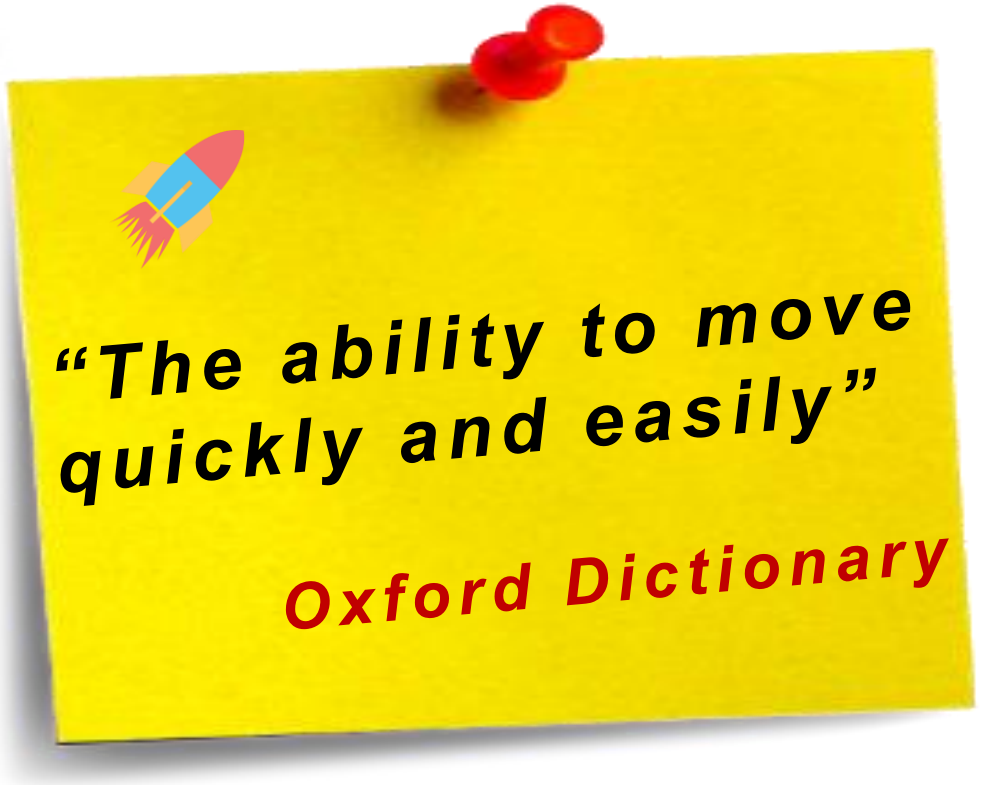
- ✓ AVAILABILITY
- ✓ SCALABILITY
- ✓ SECURITY
- ✓ PERFORMANCE
- ✓ …

VITAL QUALITY ATTRIBUTES

- ✓ MODIFIABILITY
- ✓ TESTABILITY
- ✓ FLEXIBILITY
- ✓ MAINTAINABILITY
- ✓ …

EVOLVABILITY IS A SHARED KEY INGREDIENT

AGILITY IS ...



MANIFESTO

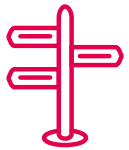
- ✓ ORGANISATION
- ✓ COLLABORATION
- ✓ PLANNING
- ✓ SOFTWARE

XP DESIGN

- ✓ MAINTAINABILITY
- ✓ TESTABILITY
- ✓ SIMPLICITY

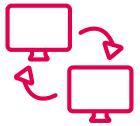
SOFTWARE EVOLVABILITY IS AT THE HEART OF AGILITY

AGILE & ARCHITECTURE



ENEMIES ? …

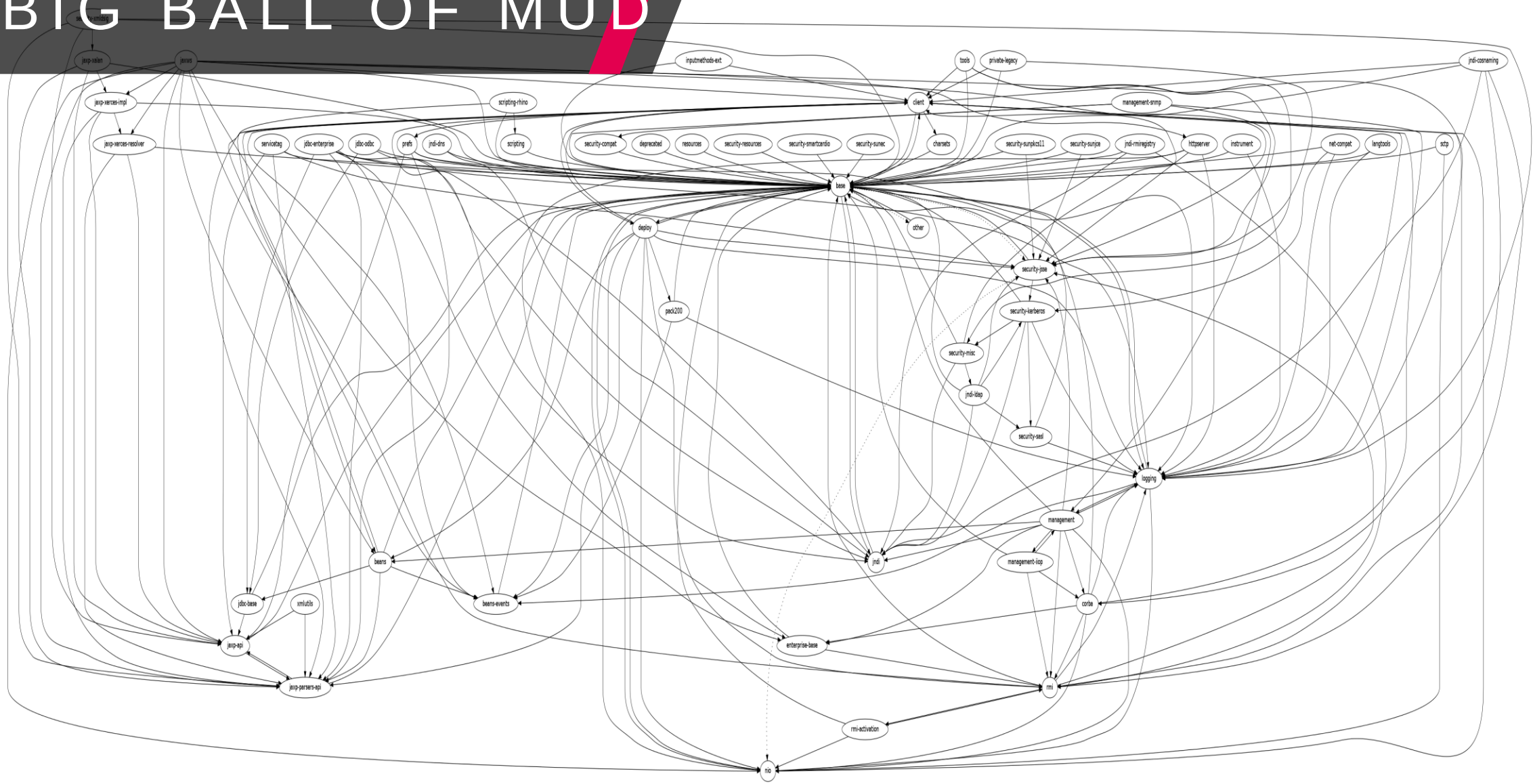
… OR FRIENDS ?



WORKING TOGETHER

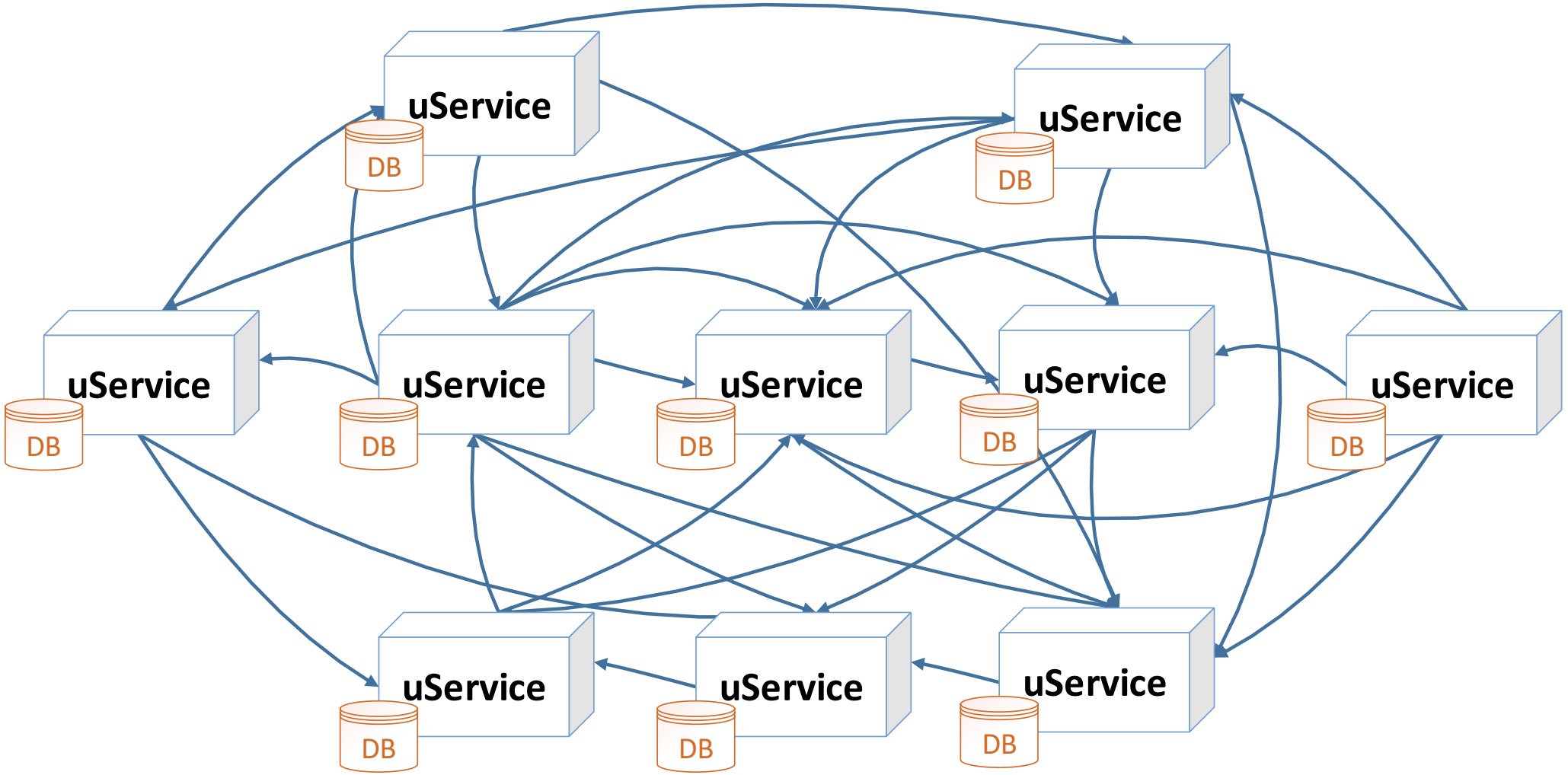
NOT AGILE ARCHITECTURES

BIG BALL OF MUD

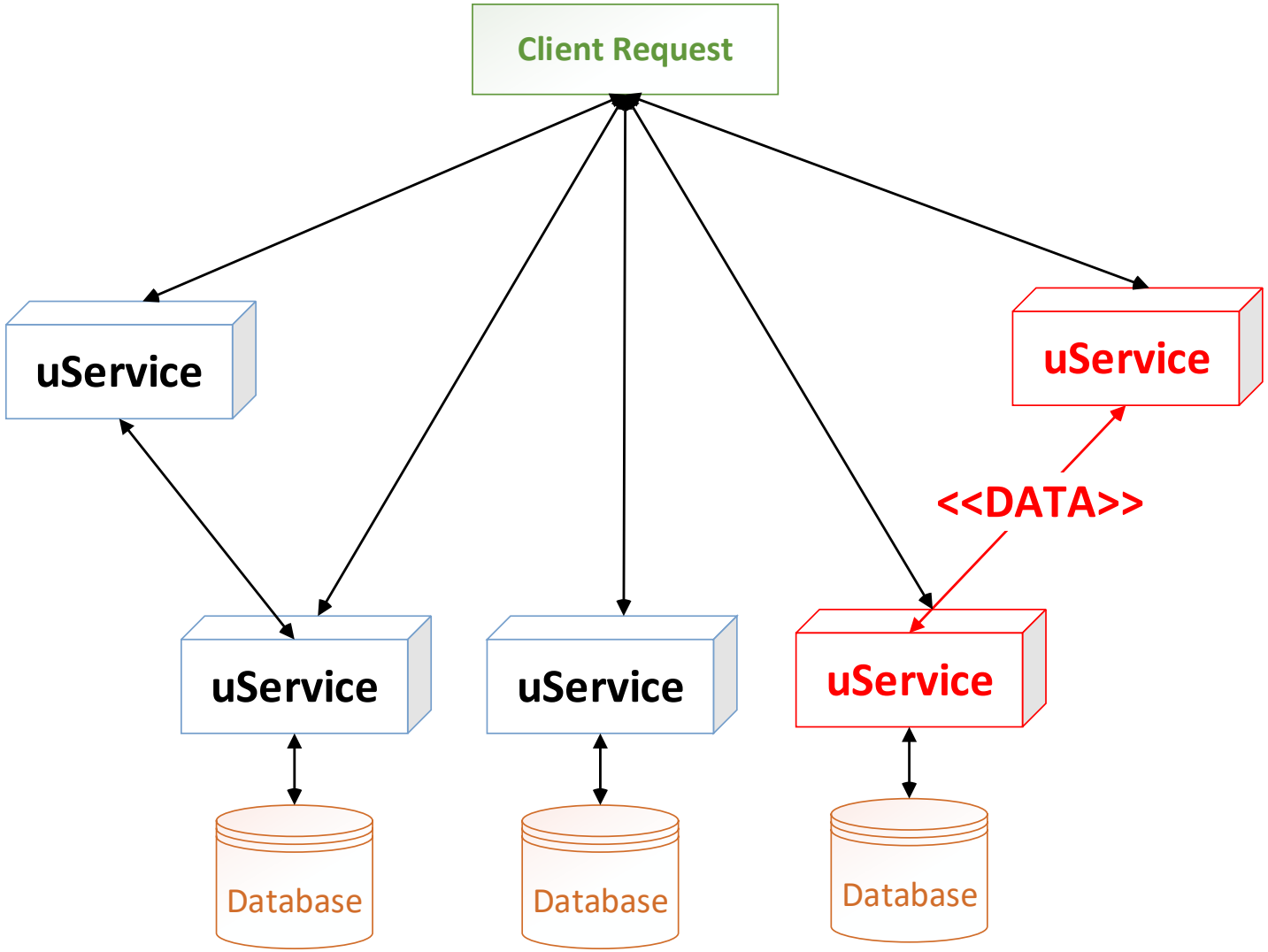


Module graph for JDK 7 b65 rt.jar, mid 2009

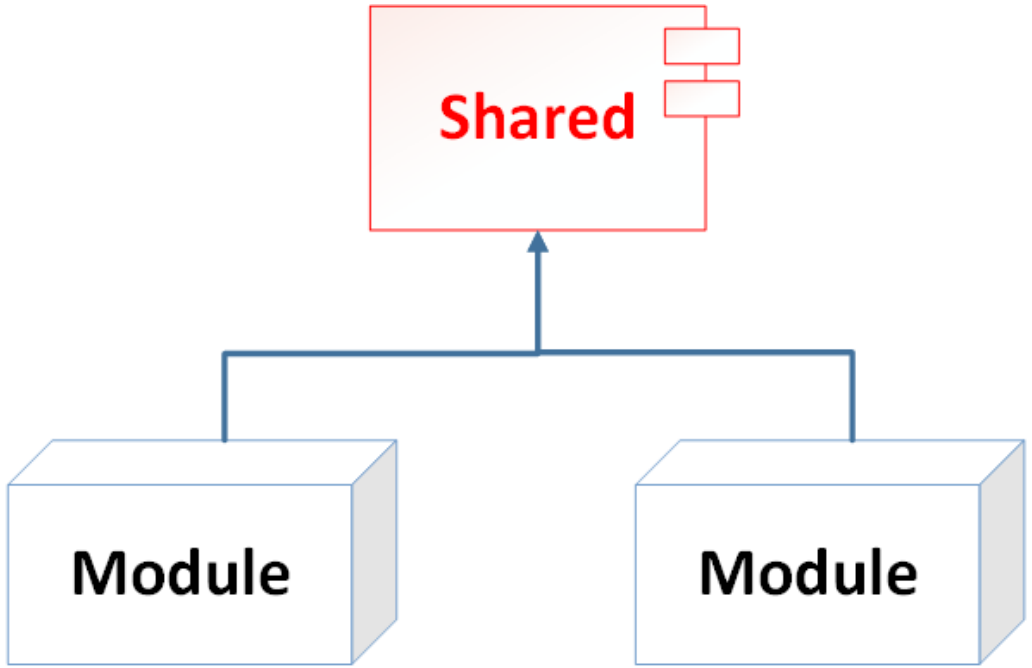
SERVICES DEPENDENCY HELL



DATA DEPENDENCY FLOWS

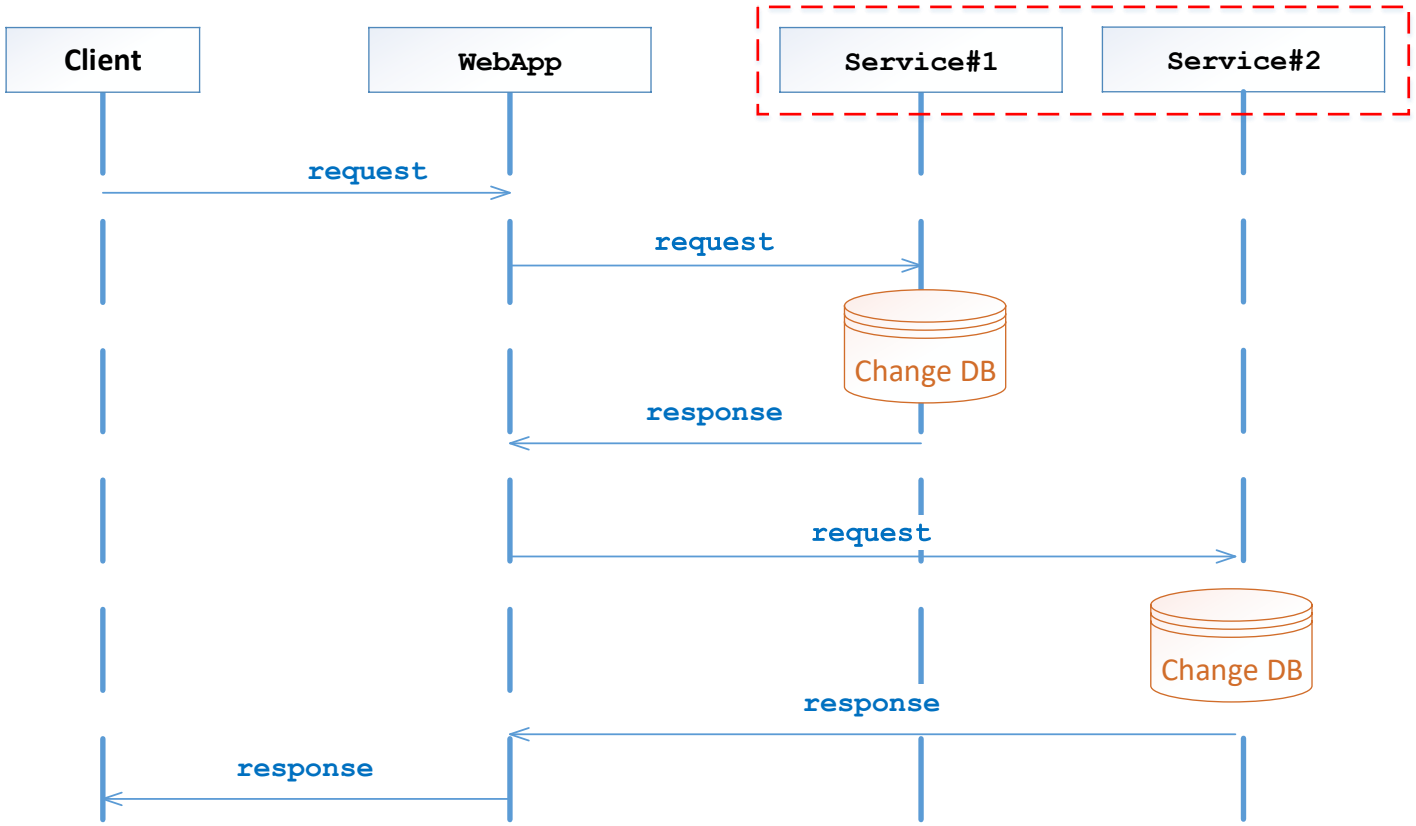


SHARED PACKAGES



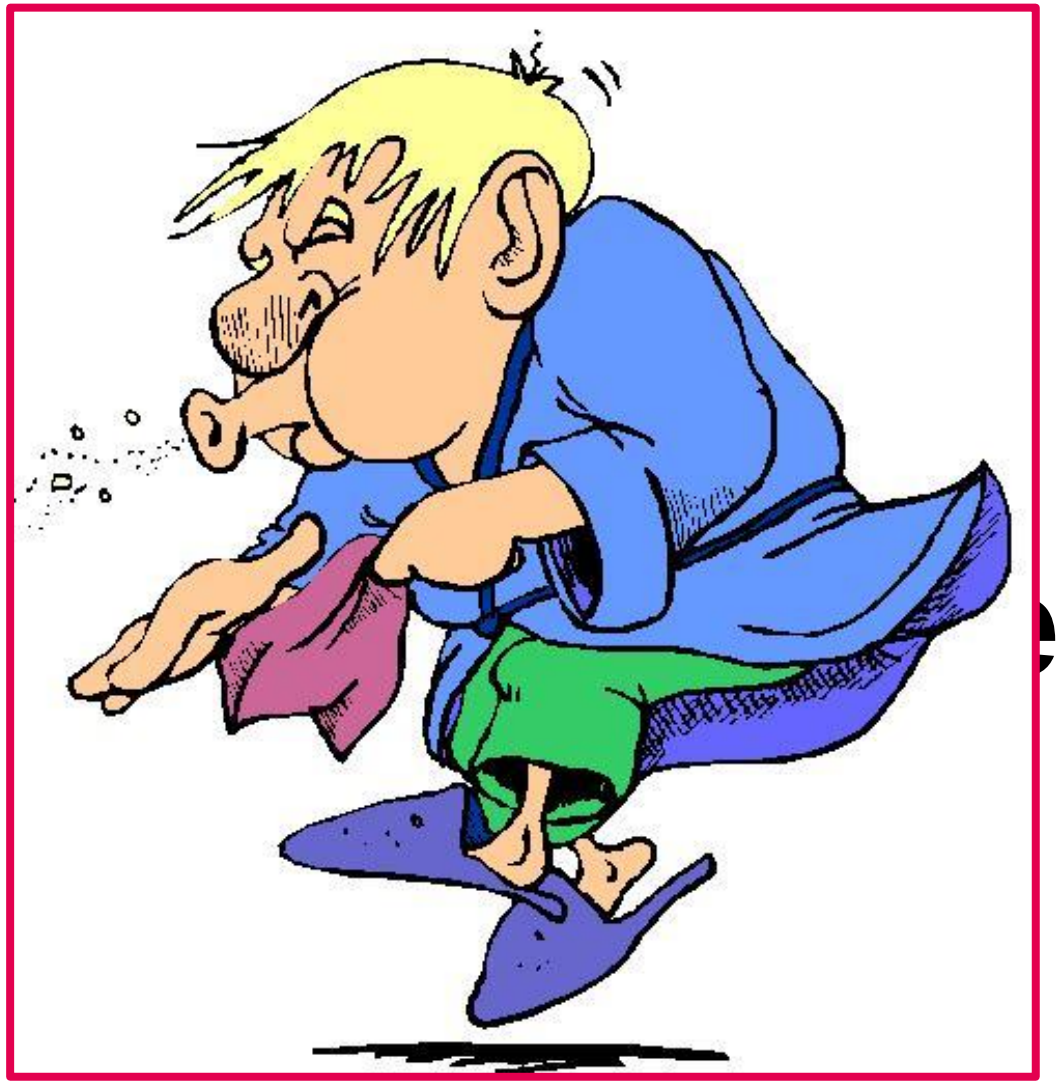
INCREASE COMPONENTS COUPLING

DISTRIBUTED TRANSACTIONS



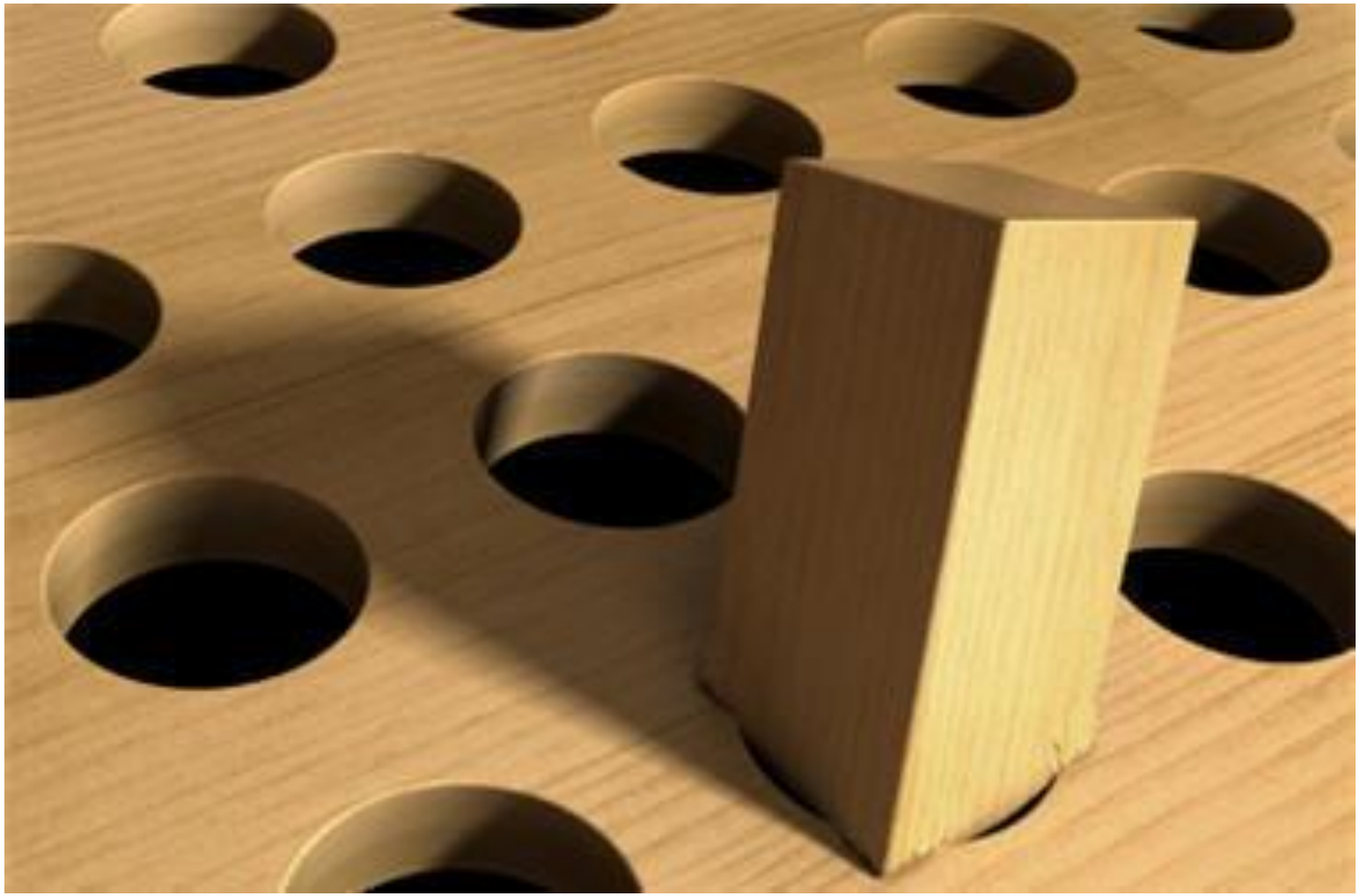
INCREASE COMPONENTS COUPLING

SYMPTOMS



es

DIFFICULT TO INCORPORATE CHANGES



SLOW RELEASE CYCLES

DIFFICULT TO CARRY OUT

SUCCESSFUL RECIPE

WHAT ARE THE INGREDIENTS FOR ACHIEVING AGILE ARCHITECTURES ?

DESIGN PRINCIPLES

KEEP IN MIND DESIGN PRINCIPLES

Do not sacrifice **loose-coupling** for performance

Prefer **composition** over inheritance

Design **messages atomic** and **services stateless**

Design remote interfaces **coarse-grained**

High cohesion inside modules, components

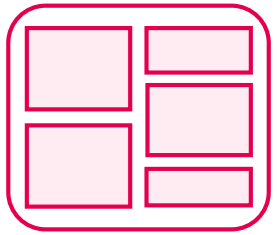
SOLID

DRY

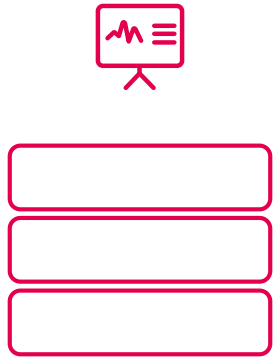
YAGNI

ARCHITECTURAL STYLES

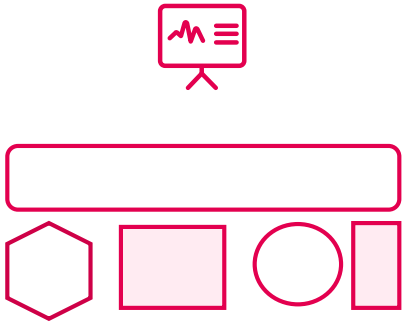
MONOLITH



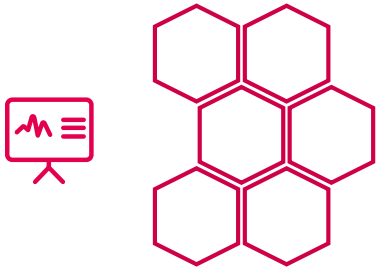
LAYERED



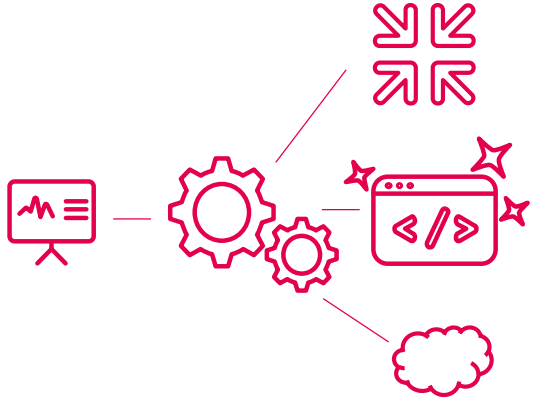
SOA



uSERVICES



SERVERLESS



COMPONENT BOUNDARIES

COMPONENT

communication mechanisms with external

protocols

synchronisation type

- synchronous
- asynchronous

transmission way

- one way
- bi-directional

messages exchanged

type

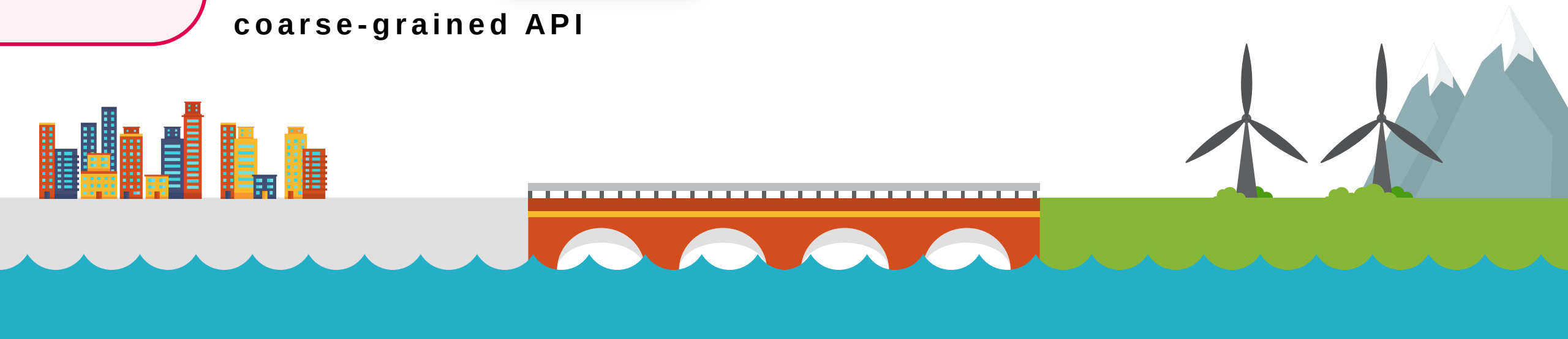
format

dependencies

depends on

used by

coarse-grained API



COMPONENT INTERNALS

COMPONENT

HIGH COHESION

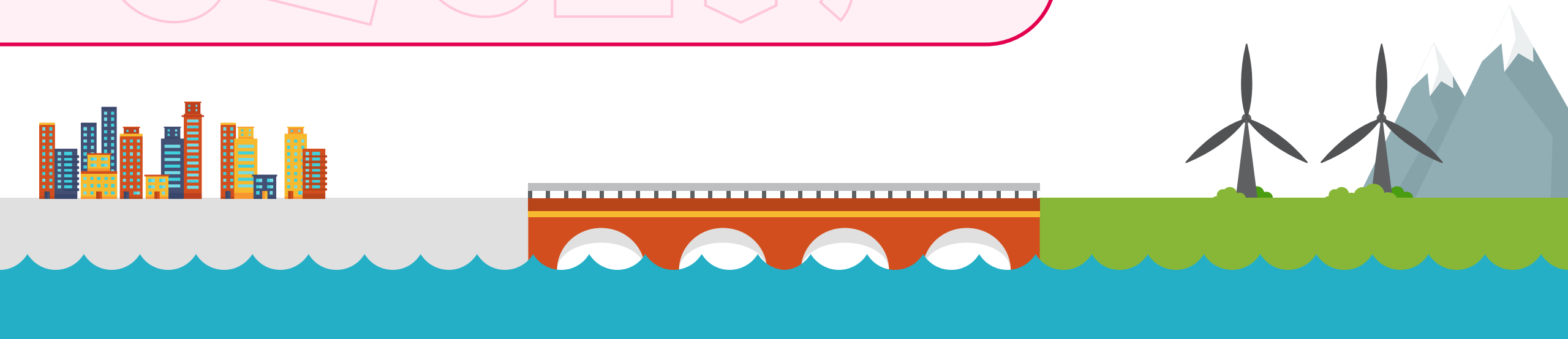
LOOSE COUPLING

ENCAPSULATION

SOLID

DRY

YAGNI



OTHER PERSPECTIVES

MANAGEMENT OF RESOURCES

Time awareness

Threading model

Scheduling strategies

Resource limits / saturation

OBJECT AND DATA MODEL

Data models

Data entities access levels

Criteria for data retention / preservation

BINDING TIME DECISIONS

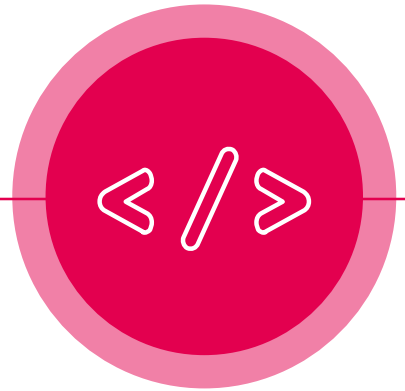
Compile time

Build time

Load time

Run time

PAY ATTENTION

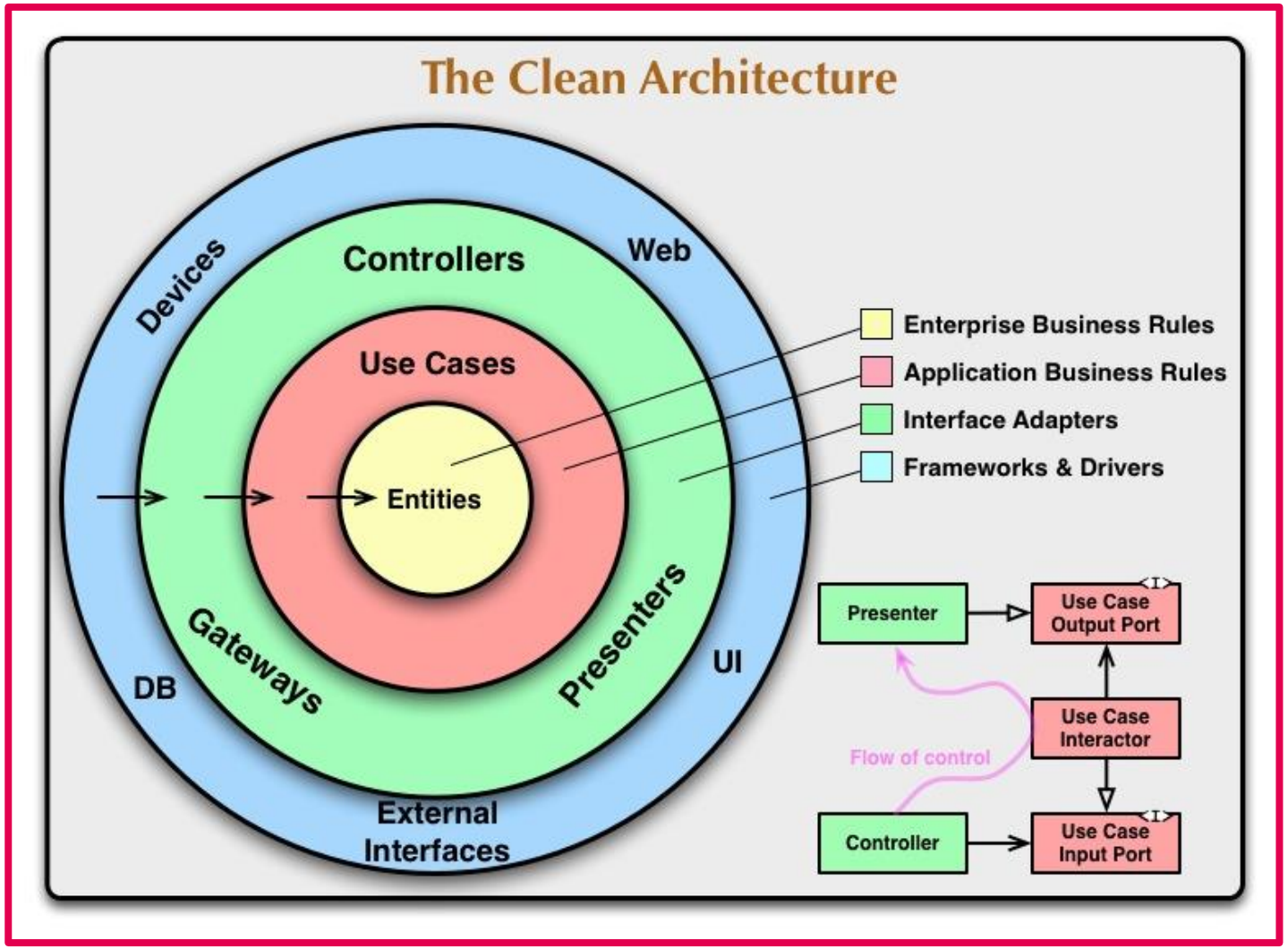


PROGRAMMING LANGUAGES

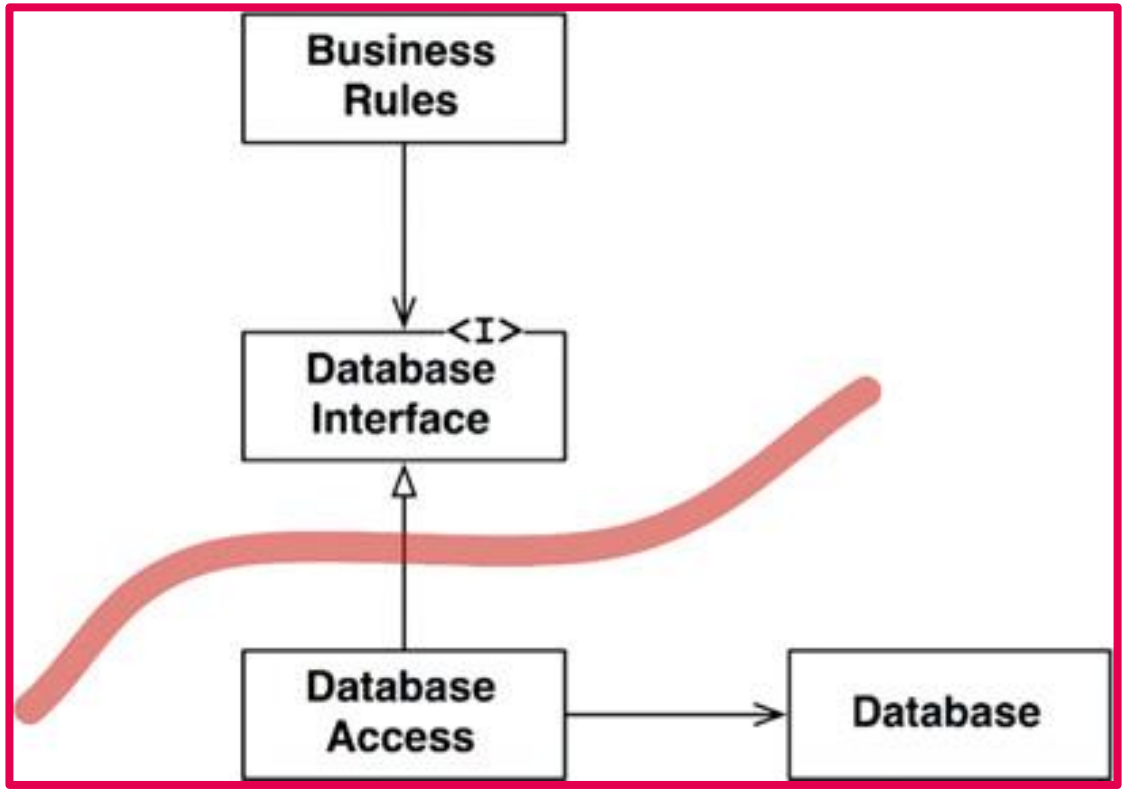
FRAMEWORKS

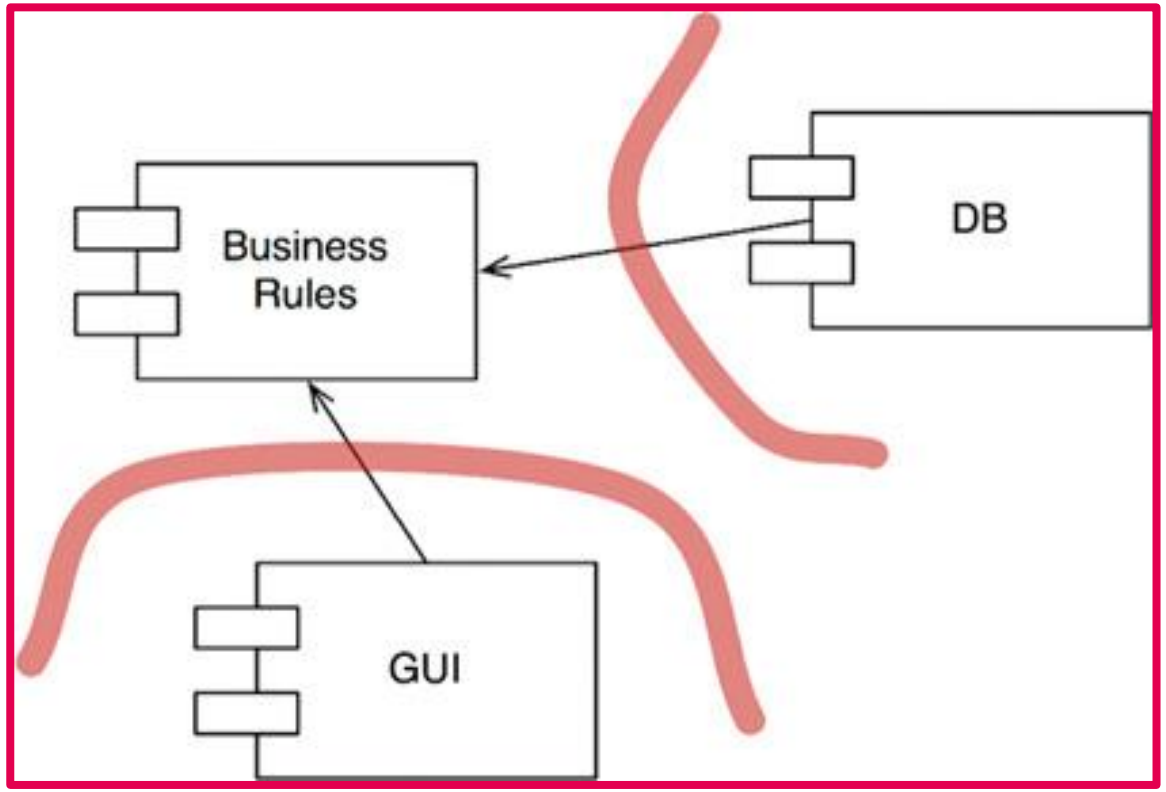
THESE MIGHT BECOME IREVERSIBLE DECISSIONS

THE DEPENDENCY RULE

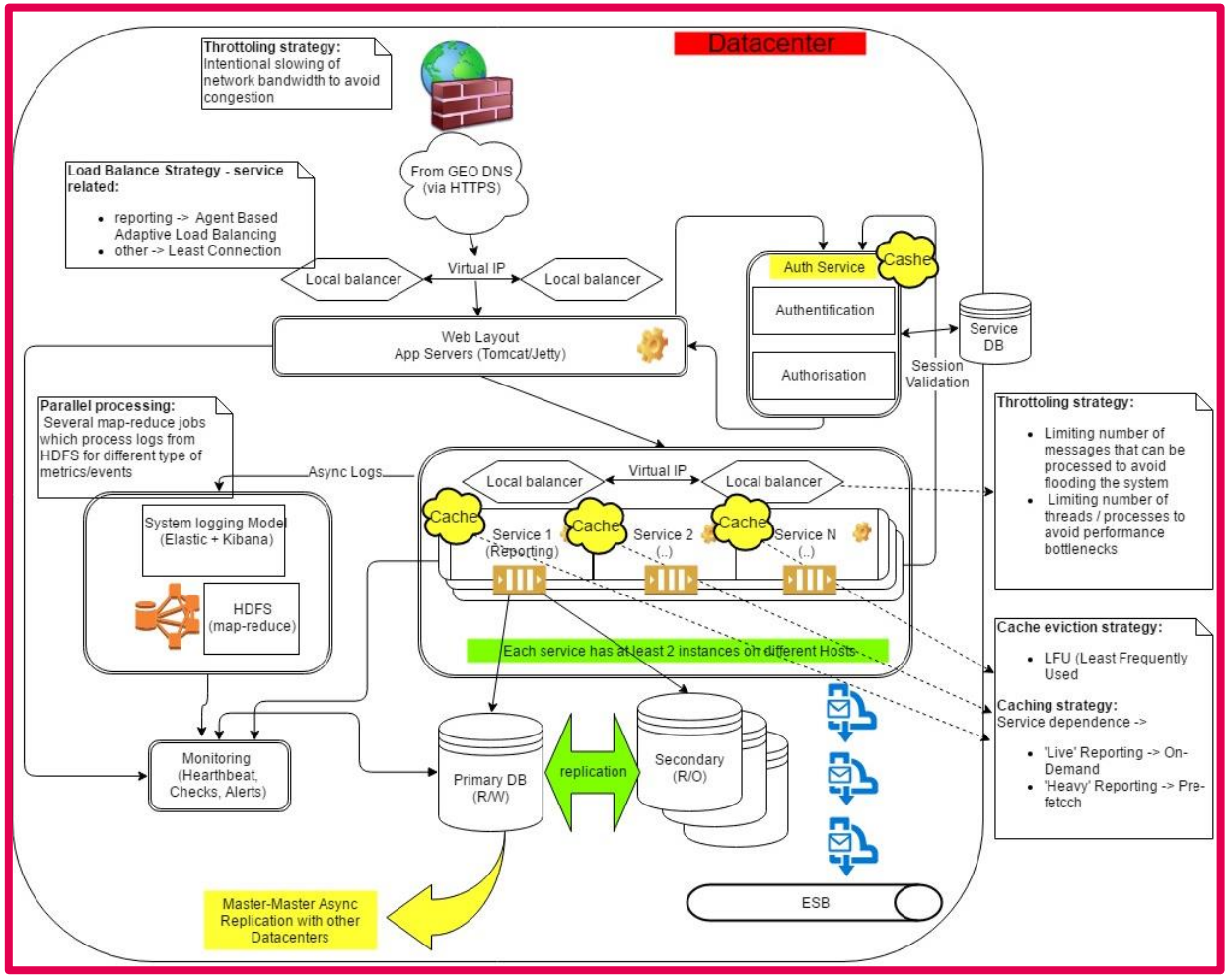


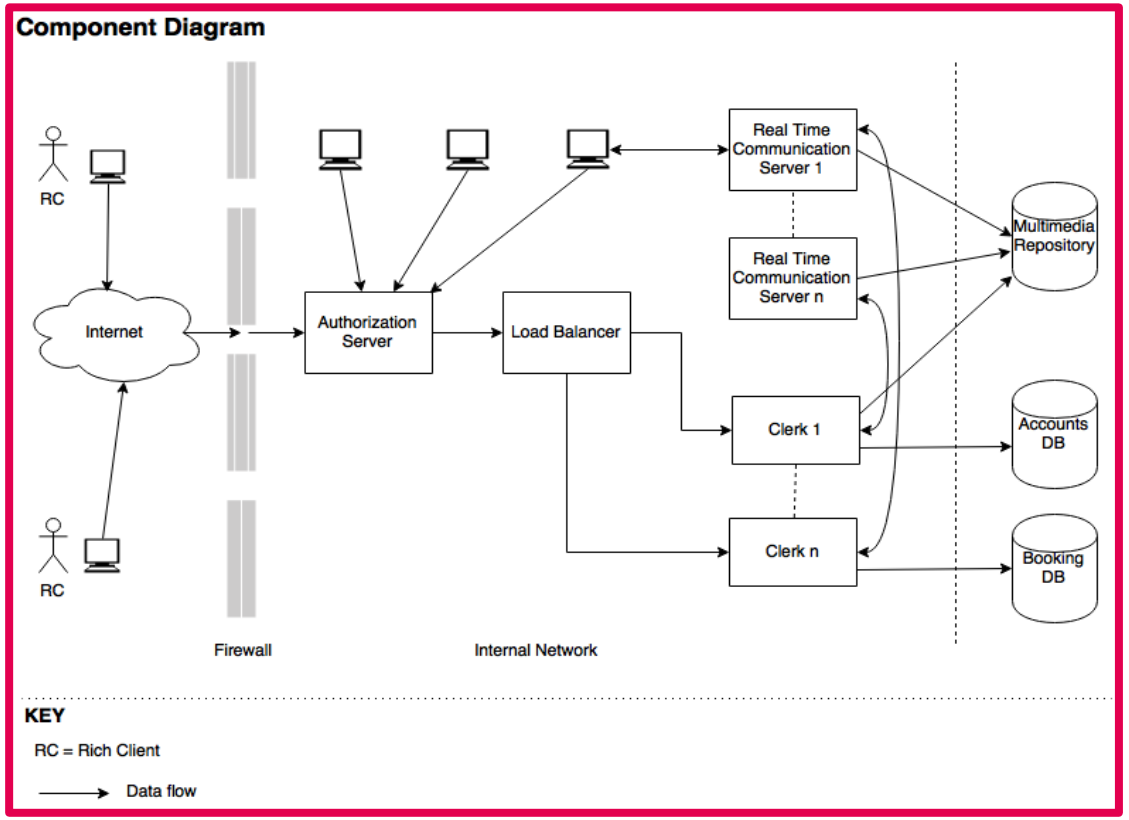
PROPER SERVICES DEPENDENCIES





WHICH ONE IS AN AGILE ARCHITECTURE ?





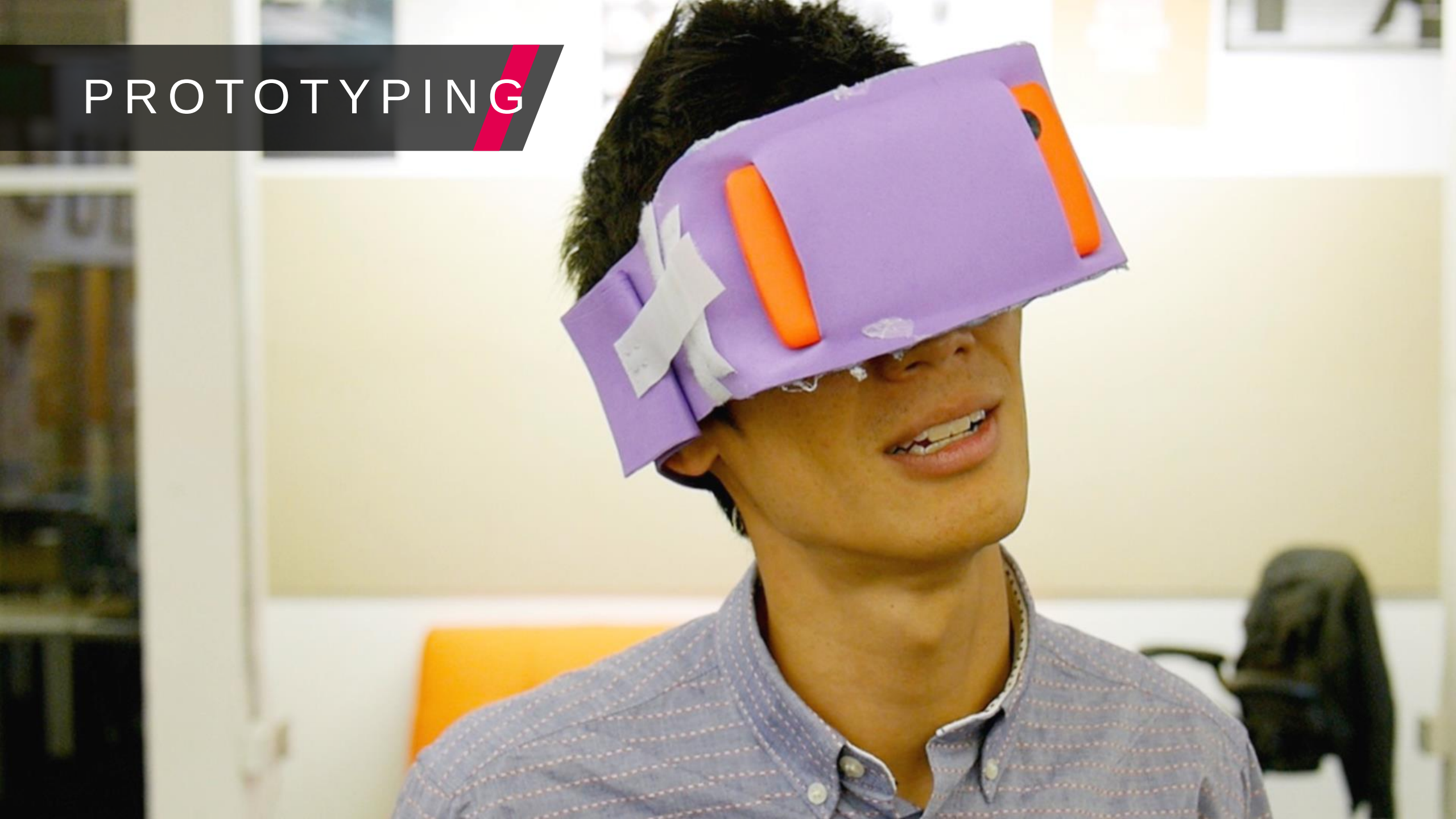
GOOD ARCHITECTURES ENABLES AGILITY

AGILITY ENABLES ARCHITECTURE DESIGN

ESSENTIALS FOR THE TRIP

W op?
A er!

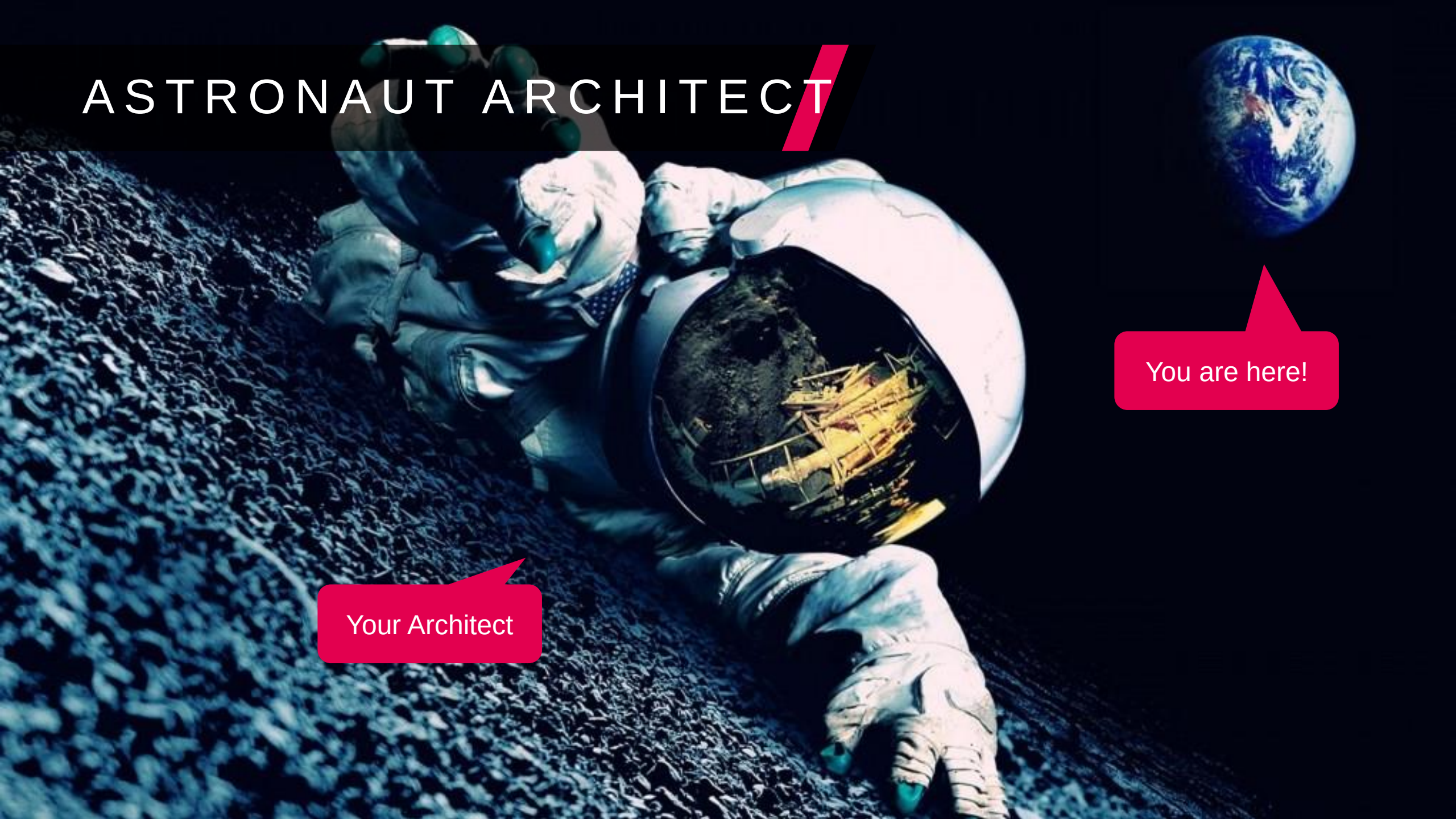
PROTOTYPING



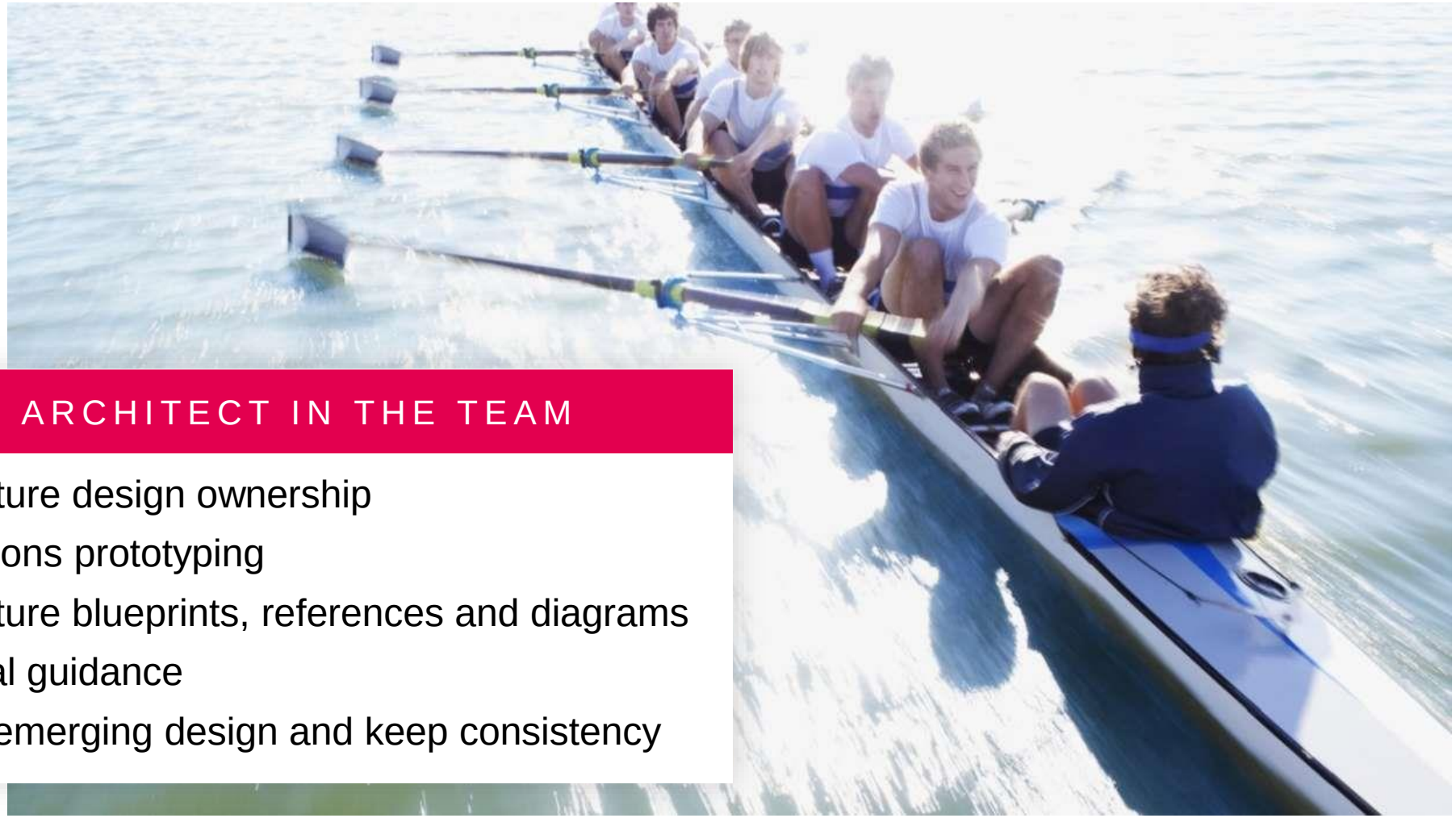

SAFE TO FAIL
✓Fitness Functions
✓Feature Toggles
✓Green / Blue Deployments

CONTINUOUS DEPLOYMENT

ASTRONAUT ARCHITECT



ALIGNMENT & AUTONOMY



THE ARCHITECT IN THE TEAM

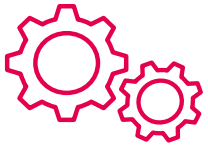
- ✓ Architecture design ownership
- ✓ Applications prototyping
- ✓ Architecture blueprints, references and diagrams
- ✓ Technical guidance
- ✓ Review emerging design and keep consistency

BLENDING AGILE AND ARCHITECTURE

REAL CHALLENGES

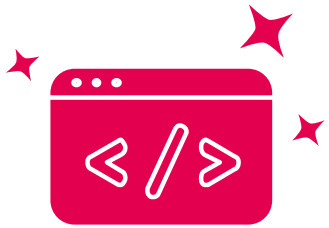
ARCHITECTURE ~ INTERNAL QUALITY → no visible external value added for Customer

Backlog is mainly driven by “Customer value” → architectural activities are not given enough attention

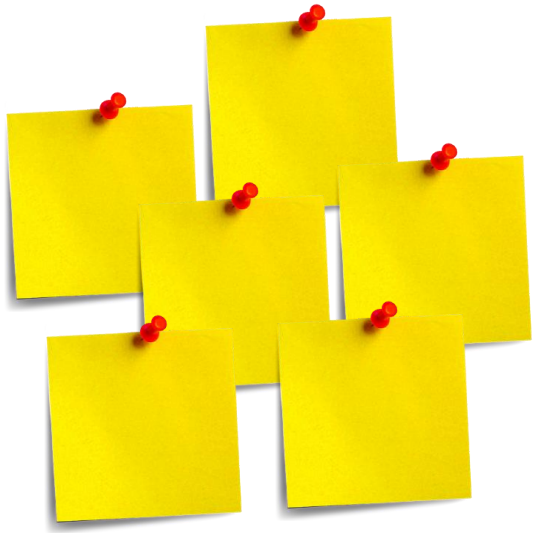


You, as ARCHITECT, ensure to make this valuable !

BACKLOG INGREDIENTS



Functionality

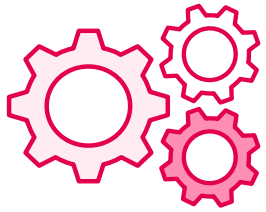


BACKLOG

Ideas

Evaluation Feedback
Technical Debt

Constraints
Life Cycle Requirements



Architecturally Significant Requirements

QUOTA RULE

70

FUNCTIONALITY

30

ARCHITECTURAL + OTHER TECHNICAL

WORKING TOGETHER BY MUTUAL AGREEMENTS ☺

THANK YOU!

@ionutbalosin

@xcorail

@Work_at_Murex

@Luxoft

https://www.linkedin.com/company/luxoft

https://fr.linkedin.com/company/murex

geecon

Krakow, 9-11 May 2018